

语法分析实验报告

语法分析程序的设计与实现（YACC 实现）实验报告

2023210992-周柄名

一、实验题目

语法分析程序的设计与实现（YACC 实现）

二、实验目的

- 理解词法分析与语法分析在编译过程中的作用与关系。
- 掌握使用 **Lex** 自动生成词法分析程序的方法。
- 掌握使用 **Yacc**（或 bison+yacc 兼容模式）自动生成自底向上语法分析程序的方法。
- 在给定算术表达式文法的基础上，实现表达式的语法分析，并在分析过程中依次输出所采用的产生式。
- 通过完整的实验流程（设计 → 编码 → 编译链接 → 测试 → 分析），提高对文法、运算符优先级及结合性的理解。

三、实验内容与要求

- 利用 YACC 自动生成语法分析程序，实现对算术表达式的语法分析。
- 要求分析的算术表达式满足如下文法（四则运算 + 括号）：

代码块

```
1  E → E + T | E - T | T
2  T → T * F | T / F | F
3  F → ( E ) | num
```

- 在对输入的算术表达式进行分析的过程中，依次输出语法分析过程中所采用的产生式。
- 使用 Lex 编写词法分析器，对输入串进行单词切分，并将记号传递给 Yacc。
- 编写实验报告，包含：
 - 实验题目、目的和要求
 - 程序设计说明（词法、语法、实现思路）
 - 源程序

- 可执行程序说明（编译命令与运行方法）
- 测试报告（输入、运行结果、分析说明）

四、实验环境

- 操作系统：Linux（或其他类 Unix 环境，Windows 下可使用 MinGW / WSL）
- 编译工具链：
 - `lex` / `flex` —— 词法分析器生成工具
 - `yacc` / `bison -y` —— 语法分析器生成工具
 - `gcc` —— C 语言编译器
- 实验源文件：
 - `expr.l` —— Lex 词法分析描述文件
 - `expr.y` —— Yacc 语法分析描述文件

注：本报告以经典的 `yacc -d` + `lex` + `gcc` 的编译流程进行说明，和实际环境中的 `bison`、`flex` 等工具兼容。

五、程序设计说明

5.1 总体设计思路

根据实验文法，将整体程序分为两个部分：

1. 词法分析器（`expr.l`）

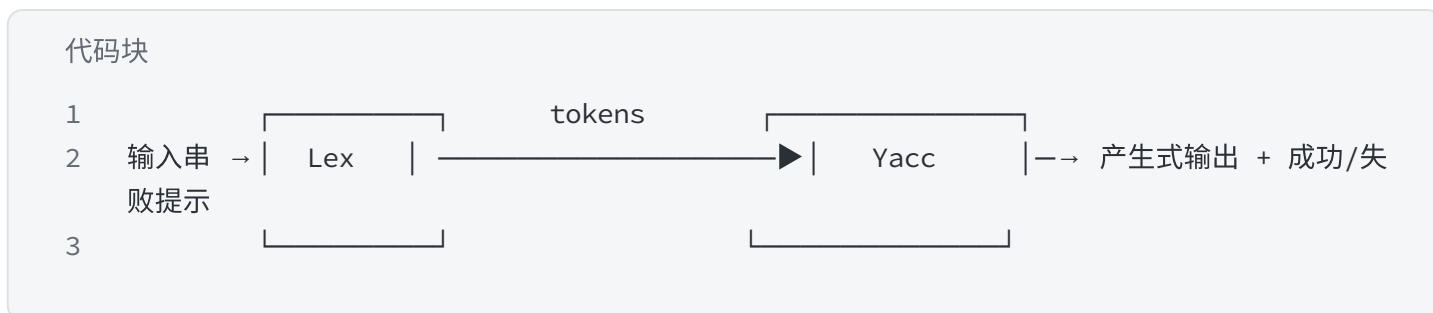
负责从输入字符流中识别记号（Token），包括数字常量 `NUM`、运算符 `+` `-` `*` `/`、括号 `(` `)` 以及跳过空白符等，并将识别到的终结符传给 Yacc。

2. 语法分析器（`expr.y`）

根据给定的文法和运算符优先级，构造表达式的语法分析器。

在每条产生式的语义动作中使用 `printf` 输出形如 `E -> E + T` 的产生式，从而满足“在分析过程中依次输出所采用的产生式”的要求。

程序整体结构如下图所示（逻辑）：



5.2 词法分析设计（`expr.l`）

5.2.1 记号设计

根据文法和需求，需要识别如下几类终结符：

- 数字常量： `NUM`
- 运算符： `+`、`-`、`*`、`/`
- 括号： `(`、`)`
- 空白字符（空格、制表符、换行）需要跳过，不作为终结符传递。

5.2.2 词法规则说明

`expr.l` 文件内容如下（与实际代码一致，仅作参考说明）：

代码块

```
1  %{
2  #include "y.tab.h"
3  #include <stdlib.h>
4  %}
5
6  %%
7
8  [0-9]+      {
9              yylval = atoi(yytext);
10             return NUM;
11         }
12
13  [ \t\r\n]+ ;
14
15  "("         return '(';
16  ")"         return ')';
17  "+"         return '+';
18  "-"         return '-';
19  "*"         return '*';
20  "/"         return '/';
21
22  .           return yytext[0];
23
24  %%
25
26  int yywrap(void) {
27      return 1;
28  }
```

说明：

1. 头部 `%{ ... %}` 中包含的 C 代码会被直接拷贝到生成的 `lex.yy.c` 中：
 - `#include "y.tab.h"`：包含 Yacc 生成的头文件，里面定义了 `NUM` 等记号和 `yylval`。
 - `#include <stdlib.h>`：用于 `atoi` 函数。
2. 规则部分：
 - `[0-9]+`：匹配一个或多个数字。
 - 将 `ytext` 转为整数，赋值给 `yylval`，然后返回终结符 `NUM`。
 - `[ \t\r\n]+`：匹配空格、制表符、回车和换行，直接忽略（不返回记号）。
 - `"("、")"、"+"、"-","*","/"`：直接返回对应的字符作为记号。
 - `.`：匹配其他任何单个字符，直接返回该字符的 ASCII 值，以便 Yacc 能够处理或报错。
3. `yywrap`：表示输入结束，返回 1 代表不再有后续输入。

5.3 语法分析设计 (expr.y)

5.3.1 头部与记号声明

代码块

```
1  %{
2      #include <stdio.h>
3      #include <stdlib.h>
4
5      int yylex(void);
6      void yyerror(const char *s);
7  %}
8
9  /* 终结符声明 */
10 %token NUM
11
12 /* 指定运算符优先级和结合性：从低到高 */
13 %left '+' '-'
14 %left '*' '/'
```

- 在 `%{ %}` 中声明了将要用到的外部函数 `yylex`、`yyerror`，并包含了标准头文件。
- 使用 `%token NUM` 声明用户自定义的终结符。
- 使用 `%left` 指定运算符的**左结合性**及其**优先级**：
 - 第一行 `%left '+' '-'` 表示 `+` 和 `-` 具有同一优先级且为左结合。
 - 第二行 `%left '*' '/'` 比上一行优先级更高，表示乘除高于加减。

5.3.2 文法规则与产生式输出

代码块

```
1  %%
2  /* 文法规则部分 */
3
4  E   : E '+' T
5        {
6            printf("E -> E + T\n");
7        }
8      | E '-' T
9        {
10           printf("E -> E - T\n");
11       }
12     | T
13       {
14           printf("E -> T\n");
15       }
16     ;
17
18  T   : T '*' F
19        {
20           printf("T -> T * F\n");
21       }
22     | T '/' F
23       {
24           printf("T -> T / F\n");
25       }
26     | F
27       {
28           printf("T -> F\n");
29       }
30     ;
31
32  F   : '(' E ')'
33        {
34           printf("F -> ( E )\n");
35       }
36     | NUM
37       {
38           printf("F -> num\n");
39       }
40     ;
```

要点说明:

1. 文法结构

与实验要求的文法完全一致：

代码块

```
1  E → E + T | E - T | T
2  T → T * F | T / F | F
3  F → ( E ) | num
```

1. 左递归结构

`E` 与 `T` 的产生式采用左递归形式（如 `E → E + T`），可以自然地配合 Yacc 的 LALR(1) 分析表以及指定的优先级，实现左结合的运算顺序。

2. 产生式输出

每条规则的语义动作中仅调用 `printf` 打印使用到的产生式，例如当归约 `E '+' T` 时输出：

代码块

```
1  E -> E + T
```

1. 这样，Yacc 在语法分析过程中顺序进行移进/归约，当发生一次归约时就执行一次对应的 `printf`，从而满足“依次输出所采用的产生式”的实验要求。

2. 没有进行语义计算

本实验重点在于语法分析与产生式输出，因此没有对表达式的值进行计算，没有使用 `$$`、`$1` 等语义值，只是简单演示文法与文法动作之间的关系。

5.4 主程序与错误处理

代码块

```
1  %%
2
3  int main(void)
4  {
5      printf("请输入一个算术表达式，例如：\n");
6      printf("  3+4*5-6/2\n\n");
7      printf("输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。 \n\n");
8
9      if (yyparse() == 0) {
10         printf("\n语法分析成功! \n");
11     } else {
12         printf("\n语法分析失败! \n");
13     }
```

```

14     return 0;
15 }
16
17 void yyerror(const char *s)
18 {
19     fprintf(stderr, "语法错误: %s\n", s);
20 }

```

- `main` 函数中提示用户输入算术表达式，然后调用 `yyparse()` 启动语法分析。
- 当 `yyparse()` 返回 0 时说明分析成功，输出“语法分析成功！”，否则输出“语法分析失败！”。
- `yyerror` 函数是 Yacc 出错时调用的回调函数，这里简单打印“语法错误:”及错误信息字符串。

六、实现步骤与编译命令

以 Linux 环境为例，源文件为 `expr.l` 和 `expr.y`。

1. 生成语法分析器

代码块

```
1 yacc -d expr.y           # 生成 y.tab.c 和 y.tab.h
```

1. 生成词法分析器

代码块

```
1 lex expr.l               # 生成 lex.yy.c
```

1. 编译并链接

代码块

```
1 gcc y.tab.c lex.yy.c -o expr
```

1. 运行程序

代码块

```
1 ./expr
```

1. 按提示输入算术表达式，回车，然后按相应的结束输入快捷键（Linux/mac 为 `Ctrl+D`，Windows 为 `Ctrl+Z` 后回车）。

七、测试用例与结果分析

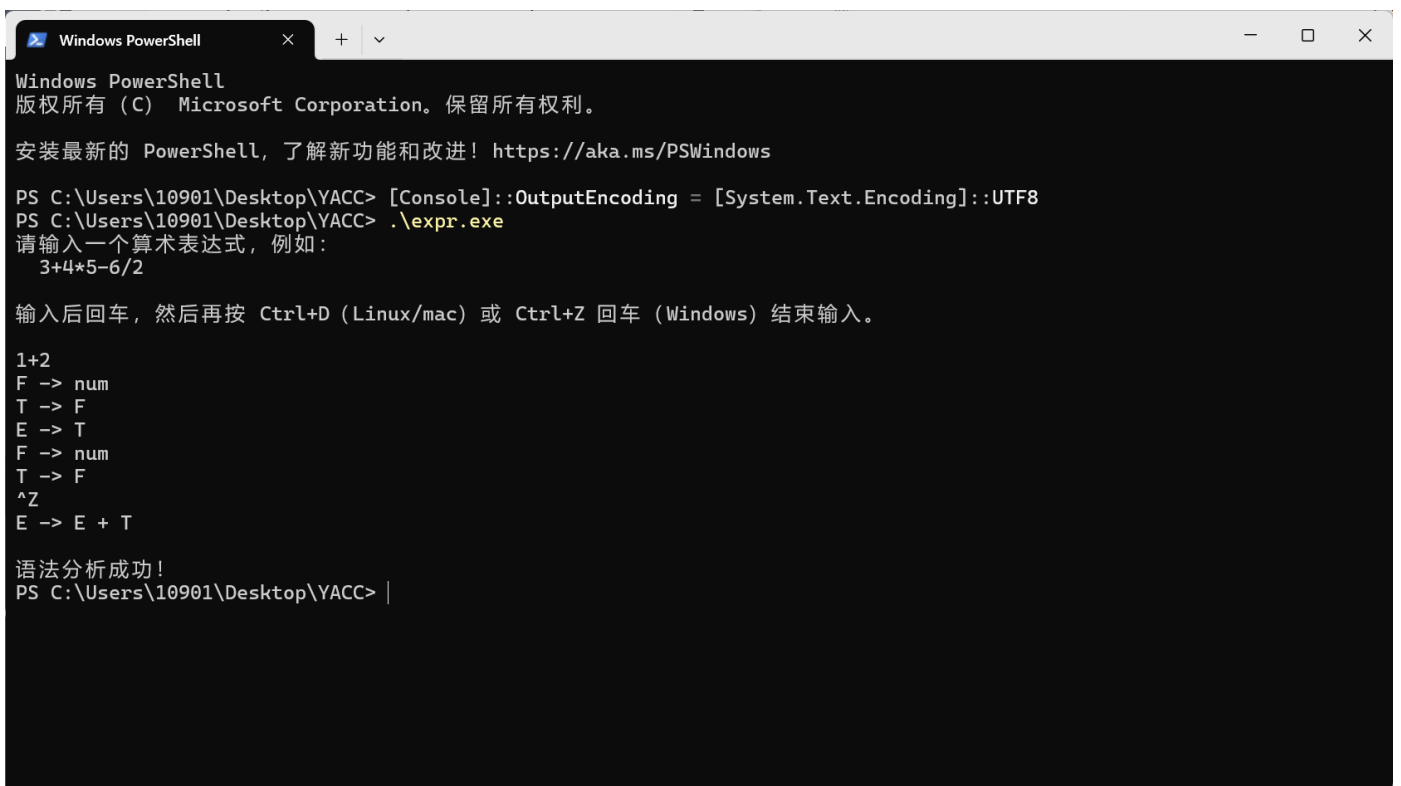
注：为了避免中文乱码问题，可以先输入 `[Console]::OutputEncoding = [System.Text.Encoding]::UTF8`

7.1 测试用例设计

选取几类典型表达式进行测试：

1. 简单加法

- 输入： `1+2`
- 目的：验证最基本的 `E -> E + T` 形式。



```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

PS C:\Users\10901\Desktop\YACC> [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
PS C:\Users\10901\Desktop\YACC> .\expr.exe
请输入一个算术表达式，例如：
3+4*5-6/2

输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。

1+2
F -> num
T -> F
E -> T
F -> num
T -> F
^Z
E -> E + T

语法分析成功！
PS C:\Users\10901\Desktop\YACC> |
```

2. 包含乘法优先级

- 输入： `3+4*5`
- 目的：验证乘法高于加法的优先级设置是否正确，期望先对 `4*5` 进行归约，再进行加法归约。

```
PS C:\Users\10901\Desktop\YACC> .\expr.exe
```

请输入一个算术表达式，例如：

3+4*5-6/2

输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。

3+4*5

F -> num

T -> F

E -> T

F -> num

T -> F

F -> num

T -> T * F

^Z

E -> E + T

语法分析成功！

3. 多种运算符混合

- 输入：3+4*5-6/2
- 目的：同时验证加减乘除的优先级与左结合性，观察产生式输出顺序。

```
PS C:\Users\10901\Desktop\YACC> .\expr.exe
```

请输入一个算术表达式，例如：

3+4*5-6/2

输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。

3+4*5-6/2

F -> num

T -> F

E -> T

F -> num

T -> F

F -> num

T -> T * F

E -> E + T

F -> num

T -> F

F -> num

T -> T / F

^Z

E -> E - T

语法分析成功！

4. 含括号的表达式

- 输入：(1+2)*3
- 目的：验证括号改变运算顺序的情况，检查 `F -> ( E )` 的归约过程。

```
PS C:\Users\10901\Desktop\YACC> .\expr.exe
```

请输入一个算术表达式，例如：

3+4*5-6/2

输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。

(1+2)*3

F -> num

T -> F

E -> T

F -> num

T -> F

E -> E + T

F -> (E)

T -> F

F -> num

T -> T * F

^Z

E -> T

语法分析成功！

5. 非法输入

- 输入：1+*2
- 目的：测试错误处理，期望调用 `yyerror` 并输出“语法分析失败”。

```
PS C:\Users\10901\Desktop\YACC> .\expr.exe
```

请输入一个算术表达式，例如：

3+4*5-6/2

输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。

1+*2

F -> num

T -> F

E -> T

语法错误：syntax error

语法分析失败！

6. 长难输入

- 输入：3+(3*4/5+2*1)-1+((1+1)-3)

输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。

3+(3*4/5+2*1)-1+((1+1)-3)

```
F -> num
T -> F
E -> T
F -> num
T -> F
F -> num
T -> T * F
F -> num
T -> T / F
E -> T
F -> num
T -> F
F -> num
T -> T * F
E -> E + T
F -> ( E )
T -> F
E -> E + T
F -> num
T -> F
E -> E - T
F -> num
T -> F
E -> T
F -> num
T -> F
E -> E + T
F -> ( E )
T -> F
E -> T
F -> num
T -> F
E -> E - T
F -> ( E )
T -> F
^Z
E -> E + T
```

语法分析成功！

7.2 典型运行结果示例（文本示意）

以下为运行程序后可能出现的一种典型输出形式（仅示意，产生式的具体顺序由解析过程决定）：

以输入 `3+4*5-6/2` 为例：

代码块

```
1  请输入一个算术表达式，例如：
2      3+4*5-6/2
3
4  输入后回车，然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。
5
6  3+4*5-6/2
7  F -> num
8  T -> F
9  E -> T
10 F -> num
```

```
11  T -> F
12  F -> num
13  T -> T * F
14  E -> E + T
15  F -> num
16  T -> F
17  F -> num
18  T -> T / F
19  E -> E - T
20
21  语法分析成功!
```

注：以上产生式输出顺序为示意，真实运行时的顺序取决于 Yacc 生成的分析表，整体上会体现出：

- `F -> num` 用于每个数字的归约；
- `T -> T * F` / `T -> T / F` 体现乘除优先级；
- `E -> E + T` / `E -> E - T` 体现加减的左结合。

非法输入如 `1+*2` 时，程序会在分析过程中调用 `yyerror`，输出类似：

代码块

```
1  语法错误: syntax error
2  语法分析失败!
```

说明文法不能识别该串。

7.3 分析与评价

1. 文法设计与 Lex/Yacc 的结合能够很好地实现算术表达式的语法分析。
2. 使用 `%left` 声明运算符优先级和结合性，避免了大量的移进/归约冲突，使得文法设计更加简洁。
3. 通过在产生式的语义动作中添加简单的 `printf` 输出，即可清晰观察到 LR 分析过程中的归约序列，对理解自底向上分析有很大帮助。
4. 错误处理部分比较简单，仅做了统一的错误提示，如需更精确的错误位置或错误类型，还可以在 `yyerror` 中结合行号、列号等信息进行扩展。

八、实验心得与总结

1. 本实验通过一个完整的四则运算表达式语法分析器，将课堂上关于上下文无关文法、运算符优先级、左递归等概念落到了实处。

2. 在实现过程中，深刻体会到 Lex 与 Yacc 之间的接口约定（如 `yylex`、`yyparse`、`yylval`、记号值定义等），理解了词法分析与语法分析之间的分工。
3. 使用 Yacc 的过程中，需要合理指定运算符优先级和结合性，否则会出现移进/归约冲突或产生错误的语义。
4. 在产生式动作中加入调试输出是一种非常有效的手段，能够帮助我们观察语法分析过程，有助于理解 LR 分析的归约行为。
5. 本实验为后续实现带有语义计算（如表达式求值）、语义检查甚至中间代码生成的编译器各个阶段打下了基础。

九、附录：完整源程序

9.1 词法分析程序 `expr.l`

代码块

```
1  %{
2  #include "y.tab.h"
3  #include <stdlib.h>
4  %}
5
6  %%
7
8  [0-9]+      {
9              yylval = atoi(yytext);
10             return NUM;
11         }
12
13  [ \t\r\n]+ ;
14
15  "("         return '(';
16  ")"         return ')';
17  "+"         return '+';
18  "-"         return '-';
19  "*"         return '*';
20  "/"         return '/';
21
22  .           return yytext[0];
23
24  %%
25
26  int yywrap(void) {
27      return 1;
28  }
```

9.2 语法分析程序 `expr.y`

代码块

```
1  %{
2      #include <stdio.h>
3      #include <stdlib.h>
4
5      int yylex(void);
6      void yyerror(const char *s);
7  %}
8
9  /* 终结符声明 */
10 %token NUM
11
12 /* 指定运算符优先级和结合性：从低到高 */
13 %left '+' '-'
14 %left '*' '/'
15
16 %%
17 /* 文法规则部分 */
18
19 E  : E '+' T
20     {
21         printf("E -> E + T\n");
22     }
23   | E '-' T
24     {
25         printf("E -> E - T\n");
26     }
27   | T
28     {
29         printf("E -> T\n");
30     }
31   ;
32
33 T  : T '*' F
34     {
35         printf("T -> T * F\n");
36     }
37   | T '/' F
38     {
39         printf("T -> T / F\n");
40     }
41   | F
42     {
43         printf("T -> F\n");
```

```

44     }
45     ;
46
47 F   : '(' E ')'
48     {
49         printf("F -> ( E )\n");
50     }
51 | NUM
52     {
53         printf("F -> num\n");
54     }
55     ;
56
57 %%
58
59 int main(void)
60 {
61     printf("请输入一个算术表达式, 例如: \n");
62     printf("  3+4*5-6/2\n\n");
63     printf("输入后回车, 然后再按 Ctrl+D (Linux/mac) 或 Ctrl+Z 回车 (Windows) 结束输入。 \n\n");
64
65     if (yyparse() == 0) {
66         printf("\n语法分析成功! \n");
67     } else {
68         printf("\n语法分析失败! \n");
69     }
70     return 0;
71 }
72
73 void yyerror(const char *s)
74 {
75     fprintf(stderr, "语法错误: %s\n", s);
76 }

```